

---

# JOJ

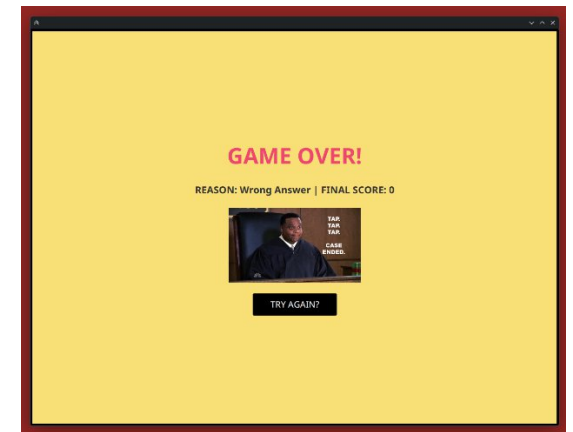
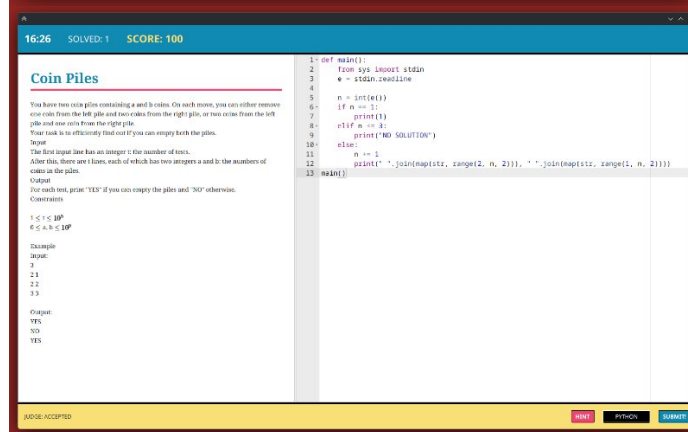
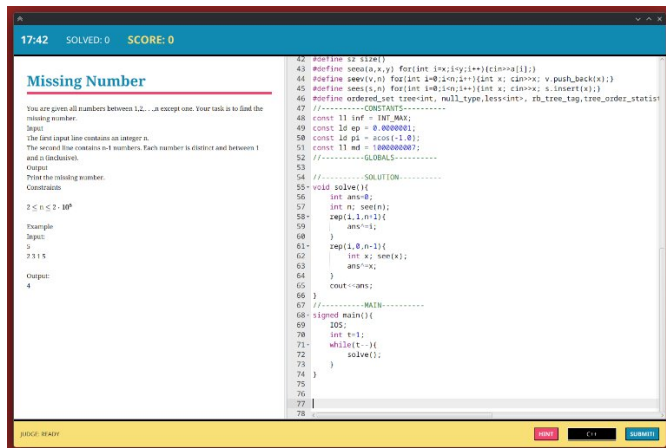
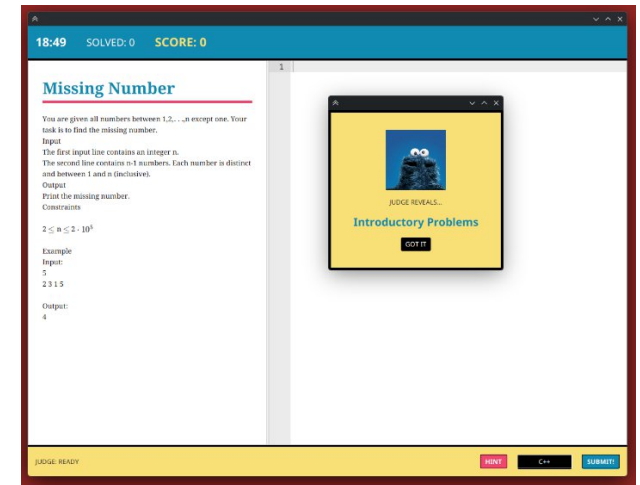
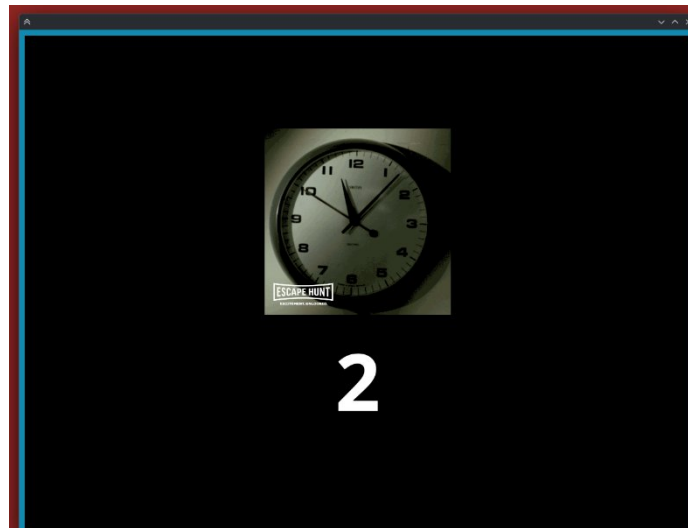
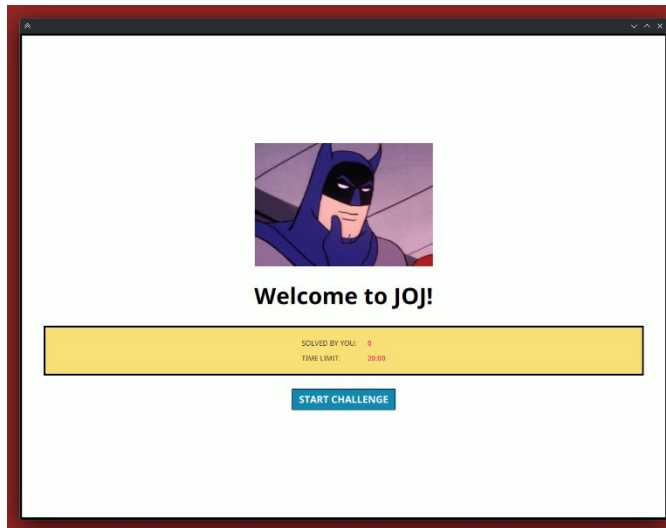
---

**Joe's Offline Judge (JOJ)** is a desktop application developed in Java that combines the technical focus of competitive programming with the high-stakes progression of a "rogue-like" game. The system presents the user with a randomized deck of coding challenges, managed by a specialized Dealer class. Each session is treated as a unique "run," where the objective is to clear as many problems as possible from a persistent pool. The application supports multiple programming languages, specifically *Python* and *C++*, and uses automated judging engines to verify code accuracy against hidden test cases. By tracking success rates and execution speeds through a local database, **JOJ** creates a competitive environment that rewards both precision and efficiency.

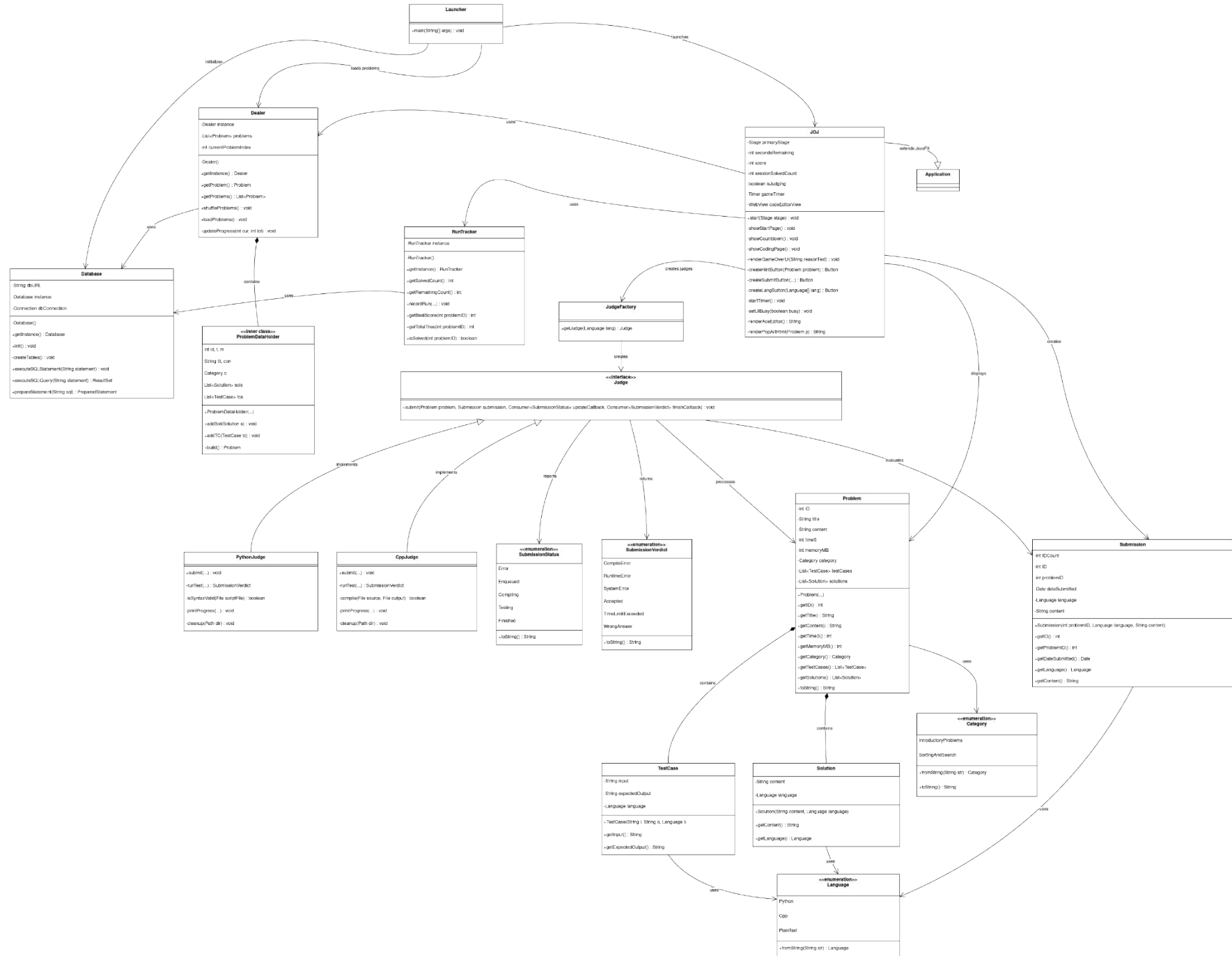
## Project Objectives

- **Problem Resolution and Survival:** The primary goal is to maximize the "Solved Count" during a single run. The *RunTracker* class interacts with the database to monitor how many unique problems the user has cleared, encouraging a high-accuracy approach where every "Accepted" verdict counts toward a new high score.
- **Modular Design (Design Patterns):** To keep the project organized and professional, the Factory Design Pattern was used. The *JudgeFactory* class handles the creation of judging objects, automatically selecting between the *PythonJudge* and *CppJudge* based on the user's input. This makes the system easy to expand with more languages in the future.
- **System Reliability (Exception Handling):** A core objective was to create a crash-proof experience. The application uses Exception Handling to manage everything from database connection errors in the *Database* class to potential issues when running the user's code in external processes. Instead of crashing, the system catches these errors and reports them safely to the interface.
- **Persistent Data (Database Integration):** While the project allows for file-based storage, it implements a fully functional **SQLite** database. This handles the long-term storage of "runs," keeping track of which problems were solved, the time taken for each, and the specific verdicts received, so a user's progress is saved across different sessions.
- **Secure Code Execution via Temp Files:** To ensure safety and isolation, the judging engines do not run code directly in the application's main folder. Instead, they use Temporary File Management to create isolated workspaces for every submission. The system writes the user's code to a temporary file, executes it in a separate process, and then cleans up the directory once the results are recorded.
- **User Experience (Fully Functional GUI):** The project features a complete graphical interface built with JavaFX. The GUI includes a countdown timer to add pressure, judging status that show real-time status, and a web-view component that renders complex mathematical symbols within the problem descriptions.
- **Advanced Code Techniques (Overloading & Overriding):** The project demonstrates core Java principles through Method Overriding and Method Overloading.

## GUI Screenshots



# UML



## Code Snippets

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>public interface Judge {     void submit(         Problem problem,         Submission submission,         Consumer&lt;SubmissionStatus&gt; updateCallback,         Consumer&lt;SubmissionVerdict&gt; finishCallback); }</pre>                                                                                                                                                                                                                                                                                                                                                                                           | <pre>workDir = Files.createTempDirectory("cpp_judge_"); File sourceFile = new File(workDir.toFile(), "solution.cpp");  String os = System.getProperty("os.name").toLowerCase(); String exeName = os.contains("win") ? "solution.exe" : "./solution.out"; File exeFile = new File(workDir.toFile(), exeName);  Files.writeString(sourceFile.toPath(), submission.getContent());</pre>               | <pre>// 1. Compile updateCallback.accept(SubmissionStatus.Compiling); if (!compile(sourceFile, exeFile)) {     updateCallback.accept(SubmissionStatus.Finished);     finishCallback.accept(SubmissionVerdict.CompileError);     return; }</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <pre>// 2. Test updateCallback.accept(SubmissionStatus.Testing); List&lt;TestCase&gt; testCases = problem.getTestCases(); SubmissionVerdict finalVerdict = SubmissionVerdict.Accepted;  for (int i = 0; i &lt; testCases.size(); i++) {     // Pass workDir to use file-based I/O     finalVerdict = runTest(exeFile, testCases.get(i),         problem.getTimeS(), workDir.toFile());      printProgress(i + 1, testCases.size(),         finalVerdict);     if (finalVerdict != SubmissionVerdict.Accepted)         break; }  updateCallback.accept(SubmissionStatus.Finished); finishCallback.accept(finalVerdict);</pre> | <pre>private boolean compile(File source, File output) throws IOException, InterruptedException {     ProcessBuilder pb = new ProcessBuilder(         "g++", "-O2", source.getAbsolutePath(), "-o",         output.getAbsolutePath()     );     Process p = pb.start();     boolean finished = p.waitFor(30,         TimeUnit.SECONDS);     return finished &amp;&amp; p.exitValue() == 0; }</pre> | <pre>JudgeFactory.getJudge(selectedLang[0]).submit(problem,     new Submission(problem.getID(), selectedLang[0],         code),     status -&gt; Platform.runLater(() -&gt;         statusLabel.setText("JUDGE: " +             status.toString().toUpperCase() + "...")),     verdict -&gt; Platform.runLater(() -&gt; {         setUIBusy(false);         if (verdict == SubmissionVerdict.Accepted) {             winSound.play();             score += 100;             sessionSolvedCount++;             showCodingPage();         } else {             renderGameOverUI(verdict.toString());         }     })      statusLabel.setText("JUDGE: " +         verdict.toString().toUpperCase()); });</pre> |